

Differentiable shortest-path computation on graphs with cycles: relaxation versus perturbation

Many alignment and structured-prediction problems reduce to shortest-path computations on a graph, yet using such solvers as differentiable layers requires gradients through a piecewise-constant optimum. The two standard remedies — smoothing and stochastic perturbation — both assume an acyclic graph; once cycles appear, smoothing can make the recursion ill-posed while perturbation pays an unknown variance cost. This internship maps that trade-off.

§1. Organization

- **Start date:** As soon as possible (flexible).
- **Localization:** IRISA, Rennes.
- **Supervision:** Romain Tavenard (Prof., Université Rennes 2, MALT Inria team, IRISA, Rennes).

§2. Context

A wide range of structured-prediction and sequence-alignment problems reduce to shortest-path computations on a graph. Dynamic Time Warping, for instance, is a shortest path on a grid graph, and many segmentation, parsing, and ranking tasks share the same combinatorial backbone. To embed such solvers inside end-to-end trainable pipelines, one must differentiate through them. This is non-trivial: the optimal path is a piecewise-constant function of the edge weights, so the exact gradient is zero almost everywhere and useless for learning.

Two paradigms produce informative gradients. *Continuous relaxation* replaces the hard min by a smooth surrogate, as in differentiable dynamic programming (Mensch & Blondel, 2018) and Soft-DTW (Cuturi & Blondel, 2017), giving deterministic but biased gradients. *Stochastic perturbation* (Berthet et al., 2020), and the closely related black-box differentiation of combinatorial solvers (Vlastelica et al., 2020), instead randomize the solver’s inputs and average, giving unbiased but higher-variance estimators that never open the solver. Crucially, the entire relaxation line assumes a *directed acyclic* graph, on which the dynamic program is a finite, well-ordered recursion.

§3. Scientific challenge

Cycles break the relaxation paradigm in a precise way. With a *hard* min, a positive-weight self-loop or cycle is harmless — it is simply never traversed by an optimal path. Under a *smoothed* min (e.g. log-sum-exp), however, the soft policy assigns non-zero probability to re-entering a cycle, so the smoothed value satisfies a *fixed-point* equation whose iteration is a contraction only when the smoothing temperature and the cycle weights are balanced; otherwise the soft cost diverges or becomes ill-defined. Existing differentiable all-pairs methods such as DataSP (Aviles-Rivero et al., 2024) smooth the Floyd–Warshall recursion but still inherit the no-negative-cycle requirement and do not characterize this regime. Perturbation-based estimators sidestep the recursion entirely — they only need a solver that returns a path — but nothing is known about how their variance scales with cycle density, or whether the smoothing bias of the relaxed route can be traded against that variance. This internship asks: **when a shortest-path graph contains cycles, under what conditions does each differentiable paradigm remain well-posed, and how do their bias, variance, and cost trade off?**

§4. Goals

Concretely, the candidate will:

1. characterize the convergence regime of smoothed shortest-path recursions on cyclic graphs as a function of smoothing temperature and cycle weights, identifying when the fixed point exists and is differentiable;
2. implement perturbation-based and black-box estimators for the same setting and quantify how their variance scales with cycle density;
3. compare the two paradigms on controlled benchmarks (isolated self-loops, single large cycles, varying cycle density, near-degenerate weights) along gradient bias, variance, runtime, and downstream alignment quality;
4. investigate hybrid estimators — e.g. relaxation on an acyclic backbone with perturbation localized to cyclic regions, or curvature-aware variance reduction (Petersen et al., 2024) for the perturbed estimator — and test them on a small end-to-end task.

Candidates should possess:

- good Python coding skills and familiarity with an autograd framework (PyTorch or JAX),
- knowledge of basic graph algorithms (shortest paths, dynamic programming),
- comfort with numerical optimization,
- ability to communicate in English in a scientific context.

Funding for a Ph.D. has already been secured: upon a successful internship, the candidate will be offered the opportunity to continue this work as a fully funded doctoral thesis. A successful candidate will develop or reinforce skills key to that continuation, namely:

- a practical and theoretical understanding of differentiable optimization,
- the ability to design controlled experimental protocols for gradient quality,
- experience implementing and benchmarking combinatorial solvers as differentiable layers.

§5. Bibliography

- Aviles-Rivero, A. I., et al. (2024). *DataSP: A Differentiable All-to-All Shortest Path Algorithm for Learning Costs and Predicting Paths with Context*. arXiv:2405.04923.
- Berthet, Q., Blondel, M., Teboul, O., Cuturi, M., Vert, J.-P., & Bach, F. (2020). *Learning with Differentiable Perturbed Optimizers*. NeurIPS.
- Cuturi, M., & Blondel, M. (2017). *Soft-DTW: A Differentiable Loss Function for Time Series*. ICML.
- Mensch, A., & Blondel, M. (2018). *Differentiable Dynamic Programming for Structured Prediction and Attention*. ICML.
- Petersen, F., Kuehne, H., Borgelt, C., Deussen, O., & Ermon, S. (2024). *Newton Losses: Using Curvature Information for Learning with Differentiable Algorithms*. NeurIPS.
- Vlastelica, M., Paulus, A., Musil, V., Martius, G., & Rolínek, M. (2020). *Differentiation of Blackbox Combinatorial Solvers*. ICLR.